INVERTIS UNIVERSITY, BAREILLY

B.Tech CSE – Third Year

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS553)

LAB 15

Assignment 15: Subsets and Arrangements

Assignment aim: Generate subsets and arrangements using recursion/backtracking.

Experiment No. 15.1 — Print all subsets of {1, 2, 3}

1. Experiment Title: Print all subsets of {1, 2, 3}

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Subsets use include/exclude choices; arrangements use backtracking to select unused elements and undo choices.

5. Algorithm / Procedure:
At each index, first exclude the value and recurse, then include it and recurse; print the chosen values at the end.
Then compile and execute the Java program.

6. Program:

```
public class Task_01_subset_of_array{
    static int[] a = {1, 2, 3};
    static boolean[] chosen = new boolean[a.length];
    static void subsets(int i) {
        if (i == a.length) {
            System.out.print("{");
            boolean first = true;
            for (int j = 0; j < a.length; j++) {
                if (chosen[j]) {
                    if (!first) System.out.print(", ");
                    System.out.print(a[j]);
                    first = false;
                }
            }
            System.out.println("}");
            return;
        }
        chosen[i] = false;
        subsets(i + 1);
        chosen[i] = true;
        subsets(i + 1);
    }
    public static void main(String[] args) {
        subsets(0);
    }
}
```

7. Input: Fixed input values are declared in the program: the set {1, 2, 3} or letters specified in the task.

8. Output:

```
{}
{3}
{2}
{2, 3}
{1}
{1, 3}
{1, 2}
{1, 2, 3}
```

Time / Space Complexity: O(n·2^n) time including printing each subset; O(n) auxiliary space excluding output.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What is a subset?
- How many subsets does a set of n elements have?
- Why are include and exclude choices both needed?

Experiment No. 15.2 — Print arrangements of a, b and c

1. Experiment Title: Print arrangements of a, b and c

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Subsets use include/exclude choices; arrangements use backtracking to select unused elements and undo choices.

5. Algorithm / Procedure:

At each position, choose an unused letter; mark it used, recurse to the next position, then unmark it to try another choice.

Then compile and execute the Java program.

6. Program:

```
public class Task_02_Print_arrangements_of_a_b_and_c{
    static char[] letters = {'a', 'b', 'c'};
    static boolean[] used = new boolean[letters.length];
    static char[] result = new char[letters.length];
    static void arrange(int pos) {
        if (pos == letters.length) {
            System.out.println(new String(result));
            return;
        }
        for (int i = 0; i < letters.length; i++) {
            if (!used[i]) {
                used[i] = true;
                result[pos] = letters[i];
                arrange(pos + 1);
                used[i] = false;
            }
        }
    }
    public static void main(String[] args) {
        arrange(0);
    }
}
```

7. Input: Fixed input values are declared in the program: the set {1, 2, 3} or letters specified in the task.

8. Output:

abc
acb
bac
bca
cab
cba

Time / Space Complexity: O(n·n!) time including output; O(n) auxiliary space excluding output.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What is a permutation?
- Why is used[] required?
- How many arrangements exist for three distinct letters?

Experiment No. 15.3 — Print arrangements of x and y

1. Experiment Title: Print arrangements of x and y

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Subsets use include/exclude choices; arrangements use backtracking to select unused elements and undo choices.

5. Algorithm / Procedure:
Choose an unused letter for each position; recurse; restore used[] after returning so the next arrangement can be formed.
Then compile and execute the Java program.

6. Program / Queries / Commands:

```
public class Task_03_Print_arrangements_of_x_and_y{
    static char[] letters = {'x', 'y'};
    static boolean[] used = new boolean[letters.length];
    static char[] result = new char[letters.length];
    static void arrange(int pos) {
        if (pos == letters.length) {
            System.out.println(new String(result));
            return;
        }
        for (int i = 0; i < letters.length; i++) {
            if (!used[i]) {
                used[i] = true;
                result[pos] = letters[i];
                arrange(pos + 1);
                used[i] = false;
            }
        }
    }
    public static void main(String[] args) {
        arrange(0);
    }
}
```

7. Input: Fixed input values are declared in the program: the set {1, 2, 3} or letters specified in the task.

8. Output:
xy
yx

Time / Space Complexity: O(n·n!) time including output; O(n) auxiliary space excluding output.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- How many permutations of two distinct letters exist?
- What is backtracking?
- Why unmark a letter after recursion?